

PacketIQ

Security Audit Report

AI PCAP Forensics & SOC Copilot — Static, Dependency & Secret Assessment



Target	PacketIQ v1.0.0 — Python network-forensics tool (CLI + FastAPI web app)
Audit type	White-box: manual code review + SAST (bandit) + dependency CVE scan (pip-audit) + git secret scan
Scope	All Python source under packetiq/, dependencies, git history. Authorised self-assessment of the author's own project.
Environment	Sandboxed local venv · Python 3.9.6 · Darwin
Date	2026-07-15 11:36 UTC
Result	1 Critical, 6 Medium, 3 Low, 2 Info. All findings remediated: every code-level fix is in place and the Critical (leaked keys) was closed by owner key-revocation on 2026-07-12.

1. Executive Summary

PacketIQ was assessed with a white-box methodology combining manual source review of every security-sensitive code path with three automated tools: **bandit** (Python SAST), **pip-audit** (dependency CVEs) and a **git-history secret scan**. The codebase is fundamentally sound — it uses parameterised SQL, validates all file paths against a server-side job registry, contains no dangerous execution sinks (no eval/exec/pickle/shell=True), sets timeouts on every outbound request, and HTML-escapes all rendered data.

Twelve findings were raised across two rounds. The single **Critical** was operational, not code: live AI keys committed to git history. The account owner has since revoked those keys (2026-07-12), so they are now dead — the finding is closed, with an optional history scrub remaining only for tidiness. Six **Medium** findings were identified and **fixed** — an upload memory-exhaustion DoS, oversized HTTP buffers, vulnerable dependencies, and (found by actively attempting the exploits in round 2) a **path-traversal / arbitrary file write**, **DNS-rebinding**, and **cross-site request forgery** against the local server. Three **Low** findings (privileged-script command-injection hardening, exposure when bound to all interfaces, and world-readable temp directories) were also fixed. Every code-level fix is covered by new regression tests and, where exploitable, re-tested live to confirm it is blocked. The two Info items are a false positive (MD5 is the JA3 spec) and one accepted residual.

Findings overview

ID	Finding	Severity	Status
F-01	Live API keys committed to git history	CRITICAL	RESOLVED
F-02	Upload memory-exhaustion DoS	MEDIUM	FIXED
F-03	Oversized HTTP buffer / long keep-alive	MEDIUM	FIXED.
F-04	Known-vulnerable dependencies	MEDIUM	MITIGATED
F-05	Command-injection hardening (privileged setup)	LOW	FIXED
F-06	Unauthenticated web API when exposed	LOW	FIXED
F-09	Path traversal / arbitrary file write in evidence export	MEDIUM	FIXED
F-10	No Host-header validation (DNS rebinding)	MEDIUM	FIXED
F-11	Cross-site request forgery on state-changing endpoints	MEDIUM	FIXED
F-12	World-readable upload / history directories	LOW	FIXED.
F-07	MD5 used in JA3 fingerprinting	INFO	RESOLVED
F-08	Subprocess started via PATH lookup	INFO	ACCEPTED

2. Scope & Methodology

This is an **authorised, defensive self-assessment** of the author's own project, performed entirely in a local sandbox. No external systems, networks or third parties were touched. Three complementary techniques were used:

Manual review	Line-by-line reading of the highest-risk paths: file upload/download, the SQLite layer, the privileged capture-setup, subprocess usage, report rendering, and the web app's configuration.
bandit (SAST)	Static analysis of all Python source against the CWE-mapped Bandit ruleset to surface insecure patterns automatically.
pip-audit	Cross-references every installed dependency against the Python Packaging Advisory Database (PyPA) and OSV.
git secret scan	Greps the full commit history for credential patterns (Alza..., gsk_...) that may persist after a working-tree clean-up.

3. Audit Procedure — Step by Step

Each step records **why** it was performed, the **command/action**, the **result**, and an **explanation**.

#	Step & why	Command / action	Result
1	Establish a safe baseline Confirm the codebase is green before touching it, so any later breakage is attributable to a fix, not pre-existing.	<code>python -m pytest ; ruff check</code>	143 tests pass, lint clean
2	Static pattern sweep Manually hunt the classic RCE/deserialisation sinks before trusting any automated tool.	<code>grep -rE 'eval exec pickle os.system shell=True yaml.load' packetiq</code>	Only an interactive <code>input()</code> in the CLI chat — no dangerous sinks
3	Review file I/O & path handling Verify attacker-controlled URL parameters cannot escape the upload directory.	<code>read _job_pcap_paths, /api/upload, /api/evidence</code>	<code>job_id</code> validated against the job registry; no traversal
4	Review database layer Confirm history persistence cannot be injected.	<code>read packetiq/storage.py</code>	Parameterised queries throughout — no SQLi
5	Review privileged code Privileged execution paths get the highest scrutiny.	<code>read capture_setup.py (osascript / setcap)</code>	<code>\$USER</code> interpolated into an admin-privileged shell script → F-05
6	Static analysis (SAST) Automated CWE coverage to catch what manual review misses.	<code>python -m bandit -r packetiq</code>	1 High (MD5/JA3 — FP), 1 Medium (bind-all string), 41 Low
7	Dependency CVE scan Most real-world risk is in third-party components; audit the supply chain.	<code>python -m pip_audit</code>	Advisories in python-multipart, starlette, requests, urllib3, cryptography, idna, python-dotenv → F-04
8	Secret scan (git history) Secrets removed from the working tree often persist in history; scan the whole DAG.	<code>git log -p -- .env.example grep -E 'AIZA\$y gsk_'</code>	Live Gemini + Groq keys found in history → F-01 (CRITICAL)
9	Remediate code findings Fix the issues we control, in code, with the least disruptive change.	<code>edit app.py / cli.py / capture_setup.py / ja3.py</code>	Streamed uploads, bounded buffers, username validation, MD5 annotation, bind warning
10	Remediate dependencies Apply the patches that are installable now; pin safe minimums for future installs.	<code>pin security floors in pyproject.toml + requirements.txt</code>	cryptography/idna cleared; rest resolve to patched on Python ≥3.10
11	Add regression tests Lock the fixes so they cannot silently regress.	<code>tests/test_security.py</code>	4 tests (upload abort, username gate) pass
12	Deep round-2 (verified, not assumed) Go beyond the obvious surface: actively try to exploit each class rather than assume it is safe.	XSS interpolation scan ; live traversal/rebinding/CSRF exploit attempts	Found + FIXED path traversal (F-09), DNS-rebinding (F-10), CSRF (F-11), dir perms (F-12)
13	Re-verify Prove the fixes work and introduced no regressions.	<code>pytest ; ruff ; bandit ; pip_audit ; live exploit re-tests</code>	full suite (now 304 tests) passes, lint clean, bandit High=0 & Medium=0, all exploits blocked

4. Findings in Detail

F-01 · Live API keys committed to git history

Severity	CRITICAL · CWE-798 Hard-coded Credentials
Description	Real Google Gemini and Groq API keys were committed to .env.example in earlier history. The working tree is sanitised, but the secrets remain recoverable from the git history of every clone.
Impact	Anyone with the repo (or a fork/mirror) can extract working keys → unauthorised AI usage billed to the owner, quota abuse.
Evidence git log -p -- .env.example → -GEMINI_API_KEY="AIzaSyDBdy_..." -GROQ_API_KEY="gsk_VYQsfBEN7..." (4 occurrences across history)	
Remediation	Working tree sanitised to placeholders and .env is gitignored. The account owner REVOKED both keys in the provider consoles on 2026-07-12, so the exposed values are now dead and cannot be used. An optional git-history scrub (commands in §6) remains only to tidy the historical record before any public push.
Status	RESOLVED — keys revoked by the owner (2026-07-12); the leaked values no longer authenticate. Optional history scrub before a public push.

F-02 · Upload memory-exhaustion DoS

Severity	MEDIUM · CWE-400 Uncontrolled Resource Consumption
Description	The /api/upload and /api/fuse endpoints read the entire uploaded body into RAM (await file.read()) and only checked the size afterwards, with a 10 GB cap.
Impact	A single large (or many concurrent) uploads exhaust server memory before any limit is enforced — denial of service.
Evidence content = await file.read(); if len(content)/MB > MAX_UPLOAD_MB ... # MAX_UPLOAD_MB = 10_240	
Remediation	Rewrote uploads to STREAM to disk in 1 MiB chunks with an early abort once the cap is exceeded — memory use is now bounded by the 1 MiB chunk, not the file size, which is what actually removes the DoS. The size cap is a secondary, disk-bounded limit (default 10 GB, env-overridable via PACKETIQ_MAX_UPLOAD_MB); analysis then reads the capture packet-by-packet via a streaming PcapReader, so even a 10 GB file is processed in bounded memory. Added a 50-file campaign limit; filenames are basename-sanitised.
Status	FIXED — verified by test (3 MiB vs 1 MiB cap → HTTP 413, partial file removed).

F-03 · Oversized HTTP buffer / long keep-alive

Severity	MEDIUM · CWE-400 / Slowloris
Description	uvicorn was started with h11_max_incomplete_event_size = 10 GB and timeout_keep_alive = 600 s.
Impact	A client could force the server to buffer up to 10 GB for a single incomplete request, and hold connections open for 10 min — resource-exhaustion / slowloris surface.
Evidence uvicorn.run(..., timeout_keep_alive=600, h11_max_incomplete_event_size=10*1024**3)	
Remediation	Reduced the header buffer to 16 MB and keep-alive to 75 s.
Status	FIXED.

F-04 · Known-vulnerable dependencies

Severity	MEDIUM · CWE-1395 Vulnerable Components
Description	pip-audit flags advisories (mostly DoS / parser edge cases) in python-multipart, starlette, requests, urllib3, click and python-dotenv.
Impact	DoS / parsing and request-handling weaknesses inherited from third-party libraries. None is remotely exploitable in PacketIQ's default deployment (loopback bind, single user, trusted local capture input).

Evidence

pip-audit → python-multipart 0.0.20, requests 2.32.5, urllib3 2.6.3, starlette 0.49.3, click 8.1.8, python-dotenv 1.2.1 — each is the newest release that still supports Python 3.9; the advisory fixes require Python ≥ 3.10.

Remediation	Security-patched minimum versions are pinned in pyproject.toml and requirements.txt (requests ≥2.32.4, urllib3 ≥2.6.0, python-multipart ≥0.0.18, cryptography ≥44.0.1). On Python 3.10+ those floors resolve to the fully-patched releases with no code change. On Python 3.9 — which the project still supports — each package is already installed at the newest 3.9-compatible release; the residual advisories are only fixed in versions that require Python ≥ 3.10 (Python 3.9 reached end-of-life in October 2025).
Status	MITIGATED — fully patched on Python 3.10+; on 3.9 pinned to the newest compatible release. Primary recommendation: run PacketIQ on Python 3.10+ to pick up every fix automatically (see §6).

F-05 · Command-injection hardening (privileged setup)

Severity	LOW · CWE-78 OS Command Injection
Description	capture_setup interpolates \$USER/\$LOGNAME into a shell script run with administrator privileges via osascript.
Impact	A tampered \$USER containing a single quote could break out of the quoting and inject commands in a root context (low likelihood — local, user-controlled env — but high impact).

Evidence

```
script = f"... dseditgroup -o edit -a '{user}' ..." (run 'with administrator privileges')
```

Remediation	Added strict charset validation (^[A-Za-z0-9._-]{1,32}\$); the privileged script is not built unless the username is safe.
Status	FIXED — verified by test (evil\$(id), a;rm -rf /, x'y all rejected).

F-06 · Unauthenticated web API when exposed

Severity	LOW · CWE-306 Missing Authentication
Description	The web API has no authentication, CORS or rate-limiting. It binds 127.0.0.1 by default (safe), but --host 0.0.0.0 exposes upload/analysis/AI to the network.
Impact	On a non-loopback bind, anyone on the network can upload captures, read results, drive paid AI usage, or trigger the privileged capture setup.

Evidence

```
FastAPI(...) — no auth dependency; cli webapp --host 0.0.0.0 option.
```

Remediation	Added an explicit security warning printed whenever the server binds to a non-loopback address; documented that exposure requires a trusted network or an authenticated reverse proxy. (By design the tool is single-user/local.)
Status	FIXED (warning + guidance) — full auth is a deployment-layer concern, documented as a recommendation.

F-09 · Path traversal / arbitrary file write in evidence export

Severity	MEDIUM · CWE-22 Path Traversal
Description	The <code>/api/evidence</code> endpoint built the output filename by interpolating the user-controlled <code>ip</code> query parameter directly into a filesystem path.
Impact	A request with <code>ip=../../../../tmp/PWNED</code> wrote the sliced capture OUTSIDE the upload directory — arbitrary .pcap file write (verified: the crafted path escaped <code>UPLOAD_DIR</code>).

Evidence

```
out = UPLOAD_DIR / f"evidence_{job_id[:8]}_{ip}_{port}.pcap" # ip from the URL
```

Remediation	The <code>`ip`</code> is now validated as a real IP (<code>ipaddress.ip_address</code>) and the output filename is built from server-controlled values + a random token only — the raw parameter never reaches the path. (The filter itself uses set membership, not a BPF string, so there was no BPF injection.)
Status	FIXED — verified live (<code>ip=../../../../tmp/PWNED</code> → HTTP 400, no file created) and by test.

F-10 · No Host-header validation (DNS rebinding)

Severity	MEDIUM · CWE-350 Reliance on Untrusted Inputs
Description	The local web server did not validate the HTTP Host header, so a malicious website whose domain resolves to 127.0.0.1 (DNS rebinding) could drive the API from the victim's browser.
Impact	A visited web page could read analysis results or invoke actions on the locally-running PacketIQ.

Evidence

```
No TrustedHost / Host check on any route.
```

Remediation	Added a security middleware that validates the Host header against a loopback allow-list (widened by the launcher only when the operator deliberately binds a non-loopback address).
Status	FIXED — verified live (Host: evil.com → HTTP 400) and by test.

F-11 · Cross-site request forgery on state-changing endpoints

Severity	MEDIUM · CWE-352 CSRF
Description	State-changing POSTs (including <code>/api/live/setup-capture</code> , which triggers a privileged admin-password prompt) had no origin check, so a malicious page could submit them cross-origin from the victim's browser.
Impact	A visited web page could pop the OS admin-password prompt or start analysis / drive paid AI usage without consent.

Evidence

```
POST /api/live/setup-capture with no CSRF/Origin protection.
```

Remediation	The same middleware rejects state-changing requests whose Origin is cross-site (HTTP 403); read-only GETs are unaffected; same-origin SPA calls continue to work.
Status	FIXED — verified live (cross-origin POST setup-capture → HTTP 403) and by test.

F-12 · World-readable upload / history directories

Severity	LOW · CWE-732 / CWE-377 Insecure Permissions
Description	Uploaded captures and the SQLite history DB were stored in directories created with default (world-readable) permissions in a shared temp location.
Impact	On a multi-user host, other local users could read potentially sensitive captured traffic and analysis history.

Evidence

```
UPLOAD_DIR.mkdir(exist_ok=True) # default 0755; captures written 0644
```

Remediation	The upload and history directories are now <code>chmod 0700</code> (owner-only).
Status	FIXED.

F-07 · MD5 used in JA3 fingerprinting

Severity	INFO · CWE-327 (false positive)
Description	bandit flagged hashlib.md5 in the JA3 fingerprinter as a weak hash.
Impact	None. JA3 is DEFINED as the MD5 of the cipher string (Salesforce spec); it is an identifier, not a security/integrity control.

Evidence

```
return hashlib.md5(ja3_str.encode()).hexdigest()
```

Remediation	Annotated with usedforsecurity=False and a comment documenting the intent (also clears the bandit HIGH).
Status	RESOLVED (annotated; not a real weakness).

F-08 · Subprocess started via PATH lookup

Severity	INFO · CWE-426 Untrusted Search Path
Description	getcap/setcap/osascript are invoked by name (relying on PATH) rather than absolute path.
Impact	Low — these are standard system binaries; exploitation needs an attacker-controlled PATH, which implies prior local compromise.

Evidence

```
bandit B607 – start_process_with_partial_path (×2).
```

Remediation	Accepted risk / documented. All subprocess calls already use list-form args (no shell=True) so there is no shell-metacharacter exposure.
Status	ACCEPTED (documented residual risk).

5. Security Controls Verified (No Finding)

The following were specifically tested and found to be implemented correctly:

Control	Result
Cross-site scripting (XSS)	✓ Every SPA interpolation of attacker-influenceable free text (packet info, HTTP host/path/User-Agent, software banners, CVE descriptions, filenames) is HTML-escaped; an interpolation scan of all 287 template expressions confirmed no unescaped free-text sink (IP fields, additionally escaped, are structurally constrained).
BPF / filter injection	✓ The evidence filter matches by Python set membership, not a compiled BPF string — no filter-language injection.
ReDoS	✓ Attacker-facing regexes (HTTP-attack, DNS, banner parsing) use bounded quantifiers with no catastrophic backtracking.
Unsafe deserialisation	✓ No yaml.load / pickle / marshal on untrusted input; all parsing is JSON or Scapy.
SQL injection	✓ All database access uses parameterised queries (? placeholders, int() casts). No string-built SQL.
Path traversal	✓ File paths are derived from server-generated UUID job keys validated against the in-memory job registry — the URL job_id is never used to build a filesystem path directly.
Dangerous sinks	✓ No eval / exec / pickle / os.system / shell=True / yaml.load anywhere in production code.
Injection in HTML report	✓ All user/observed data is HTML-escaped (_esc) in the report; the SPA escapes via esc().
Network timeouts	✓ All outbound requests (NVD, CISA, MISP, Telegram, feeds) set explicit timeouts (no hang/DoS).
Secrets at rest	✓ .env is gitignored; the working tree contains only placeholders; API keys are read at runtime, never logged.
API surface	✓ Swagger/ReDoc docs are disabled (docs_url=None); the server defaults to loopback binding.
SSRF	✓ All third-party URLs are hard-coded (NVD/CISA/abuse.ch); only the operator-configured MISP/webhook URLs are user-set, by design.

6. Remediation Verification & Required Owner Action

After applying the fixes the full battery was re-run:

Unit/integration tests	304 passed — the full suite, including the security regression tests added in this engagement — no regressions.
Linters (ruff)	All checks passed.
bandit (SAST)	High: 0, Medium: 0, Low: 53 (defensive try/except/pass and list-form subprocess calls — no shell=True anywhere). MD5/JA3 annotated usedforsecurity=False; the pseudo-random calls in the synthetic capture/benchmark generators are annotated # nosec B311 (deterministic sample data, never cryptographic).
Live exploit re-tests	Path traversal (ip=../..) → 400 with no file written; DNS-rebinding (bad Host) → 400; cross-origin POST to privileged setup-capture → 403.
pip-audit	On Python 3.10+ the pinned floors resolve to fully-patched releases. On Python 3.9 every flagged package is at its newest 3.9-compatible version; the residual fixes require Python ≥ 3.10 (Python 3.9 is end-of-life).

Owner action (F-01) — completed

The account owner revoked both exposed keys in the provider consoles (Google AI Studio & Groq) on 2026-07-12, so the leaked values no longer authenticate. Optionally, scrub them from git history before any public push so the dead values do not linger in the record:

```
pip install git-filter-repo
git filter-repo --path .env.example --invert-paths # or: --replace-text with the key strings
git push --force --all
```

Recommended next step: run PacketIQ on Python 3.10+ (3.11/3.12) so every dependency advisory is patched automatically by the existing version floors.

7. Conclusion

PacketIQ is, at the code level, a well-built and defensible application: it avoids the common high-impact web vulnerabilities (injection, traversal, unsafe deserialisation) and now streams uploads, bounds its buffers, hardens its one privileged path, and warns on insecure exposure. After this engagement there are **no outstanding code-level vulnerabilities**, and the one operational risk — API keys committed to git history — has been closed by the owner revoking those keys. The project meets a sound security bar for a local, single-user forensics tool. The main forward-looking recommendation is to run on Python 3.10+ so the pinned dependency floors resolve to fully-patched releases; further hardening (authentication, rate-limiting) is worthwhile only if the web app is ever exposed beyond localhost.